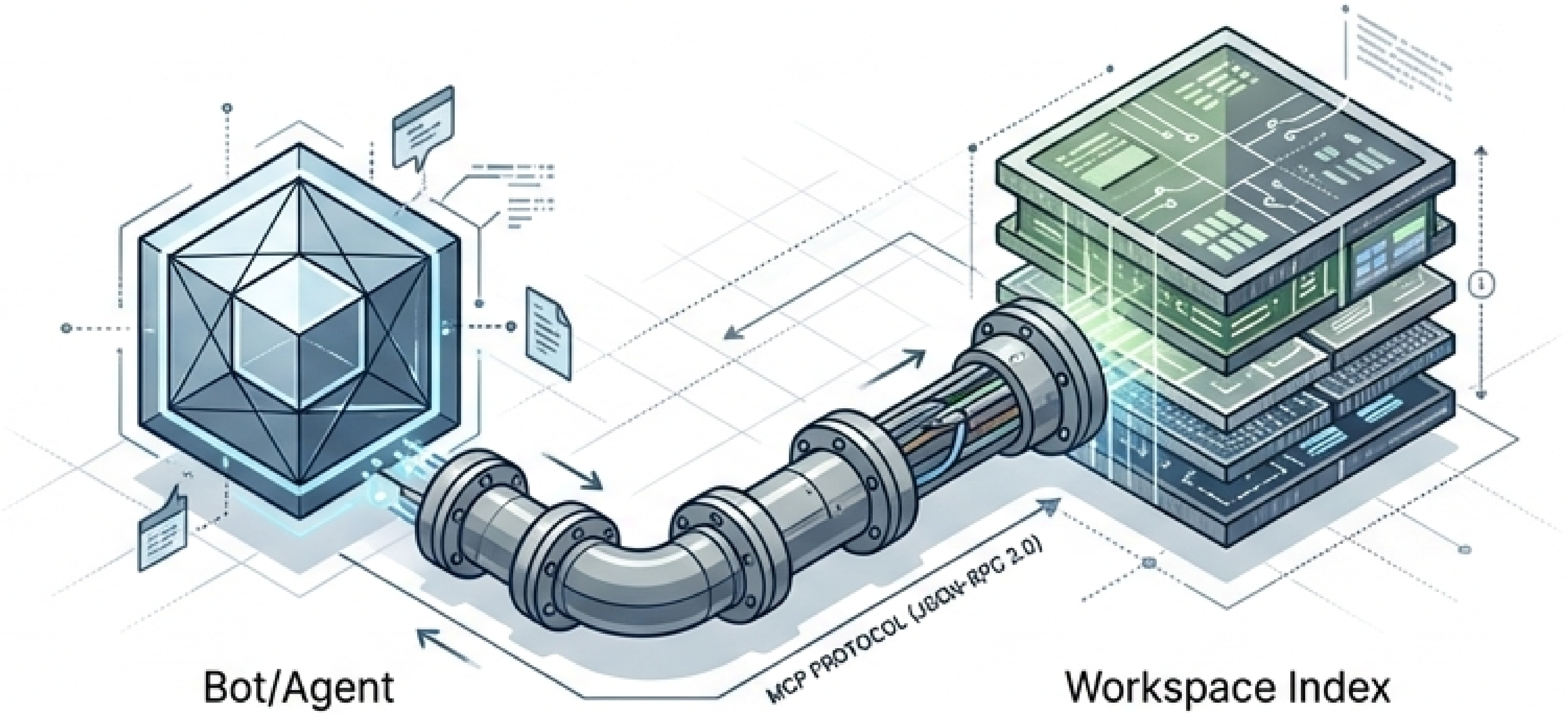
Synthesis MCP Server

Bridging AI Agents & Workspace Context

A comprehensive guide to giving Claude, Cursor, and Aider direct access to your codebase.



The Missing Link: Structured Access to Code

THE PROBLEM

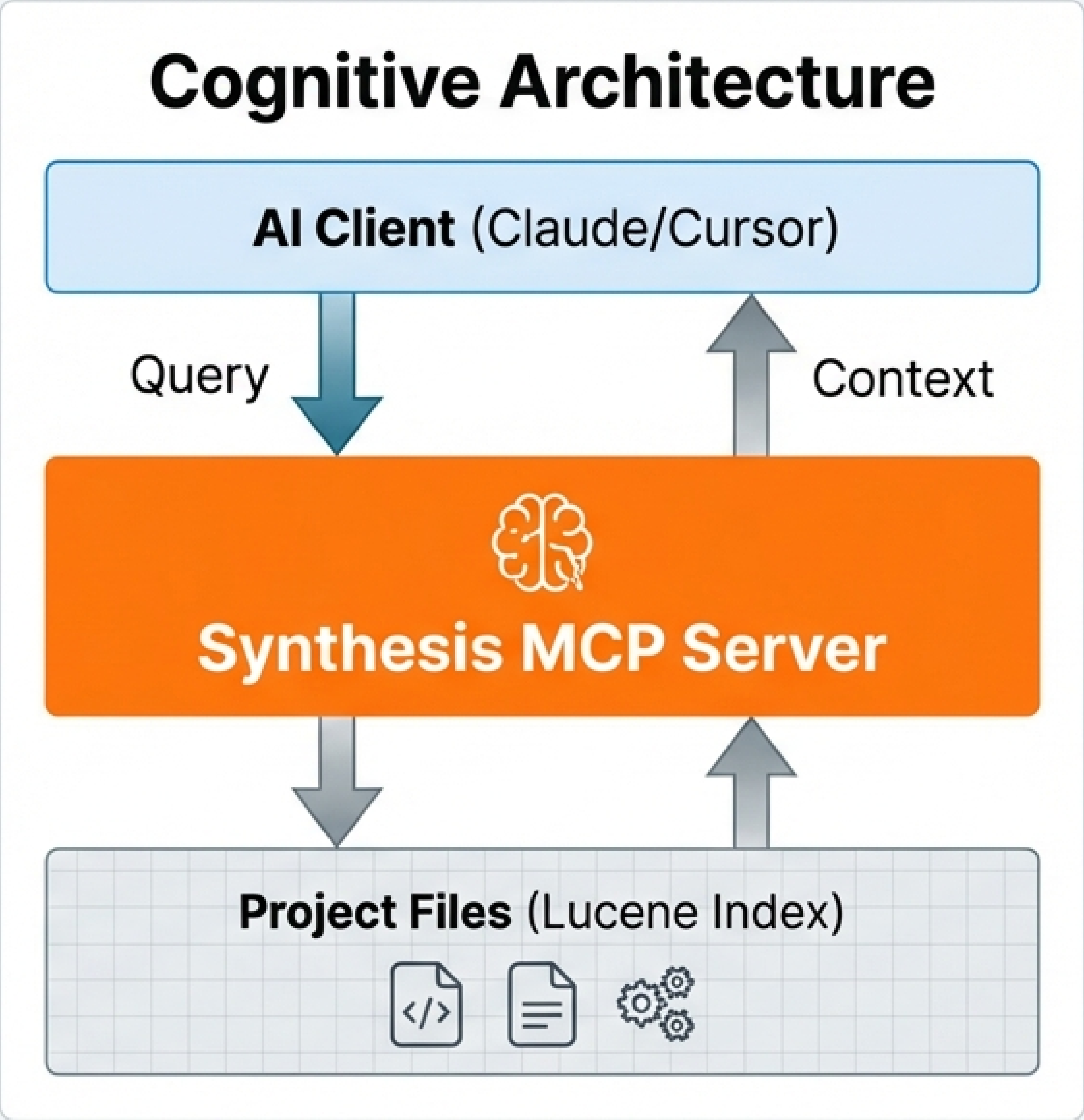
AI agents typically lack deep knowledge of large repositories. This leads to hallucinations, surface-level code generation, and a lack of awareness regarding project dependencies.

THE SOLUTION

Synthesis provides a native Model Context Protocol (MCP) implementation. It exposes a structured Lucene-based query engine to access code, configuration, and documentation.

CAPABILITIES

- Full-Text Search (Code, PDF, Config)
- Bidirectional Relationship Mapping
- Architecture Visualization
- Health Monitoring & Metadata



Installation & Initialization

01 PREREQUISITES

Ensure your environment is ready.

```
JetBrains Mono
java -version
(Must be Java 17+)

synthesis --version
(Must be 1.0.0+)
```

02 INSTALLATION

Choose your preferred method.

```
# Recommended (Installer)
curl -fsSL https://.../
install.sh | bash

# Build from Source
mvn clean package
-DskipTests

# Docker
docker run -v ...
synthesis:latest
```

03 INDEXING (VITAL)

The server requires a pre-built index.

```
JetBrains Mono
cd ~/your-project
synthesis init --name
'My Project'
synthesis scan

# Verify Status
synthesis status
```

Tool 01: Search (The Eyes)

Full-text search with Lucene precision across all file types.

LUCENE SYNTAX CHEATSHEET

Wildcard	**auth*** (Matches auth, authentication)
Boolean	testing AND strategy
Field Query	**language:Java** or **fileType:YAML**
Combined	"security policy" with **fileType**="PDF"

RESPONSE OBJECT

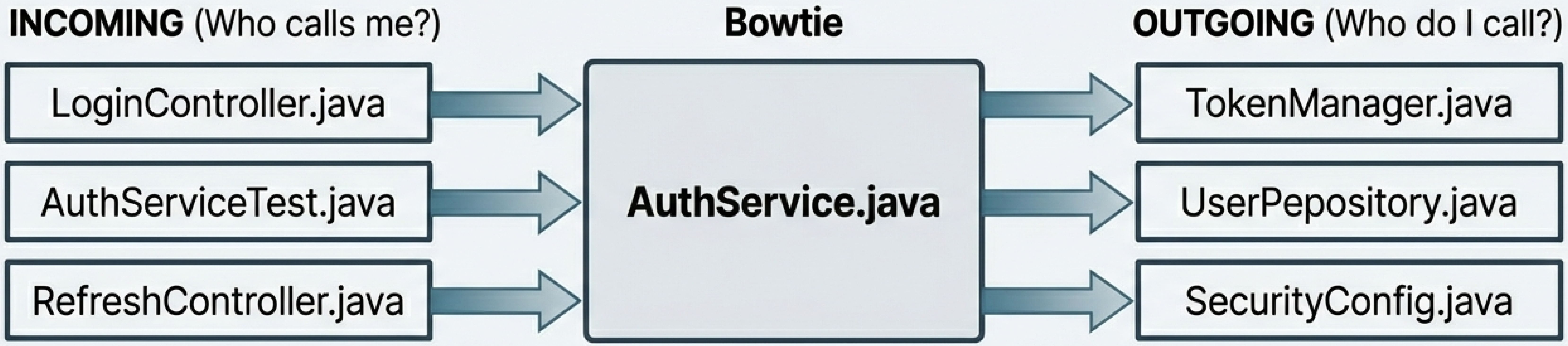
Relevance Ranking

Structured Context

```
{
  "results": [
    {
      "fileName": "AuthService.java",
      "score": 2.45,
      "type": "CODE",
      "snippet": "Authentication service handling
          OAuth2 flows...",
      "metadata": {
        "language": "Java",
        "headings": "AuthService, authenticate",
        "structure": "class AuthService { ... }"
      }
    }
  ]
}
```

Tool 02: Relate (The Brain)

Bidirectional relationship analysis to predict impact.



JSON STATS OUTPUT

```json
{
  "stats": {
    "outgoingCount": 3,
    "incomingCount": 3,
    "totalConnections": 6
  }
}
```

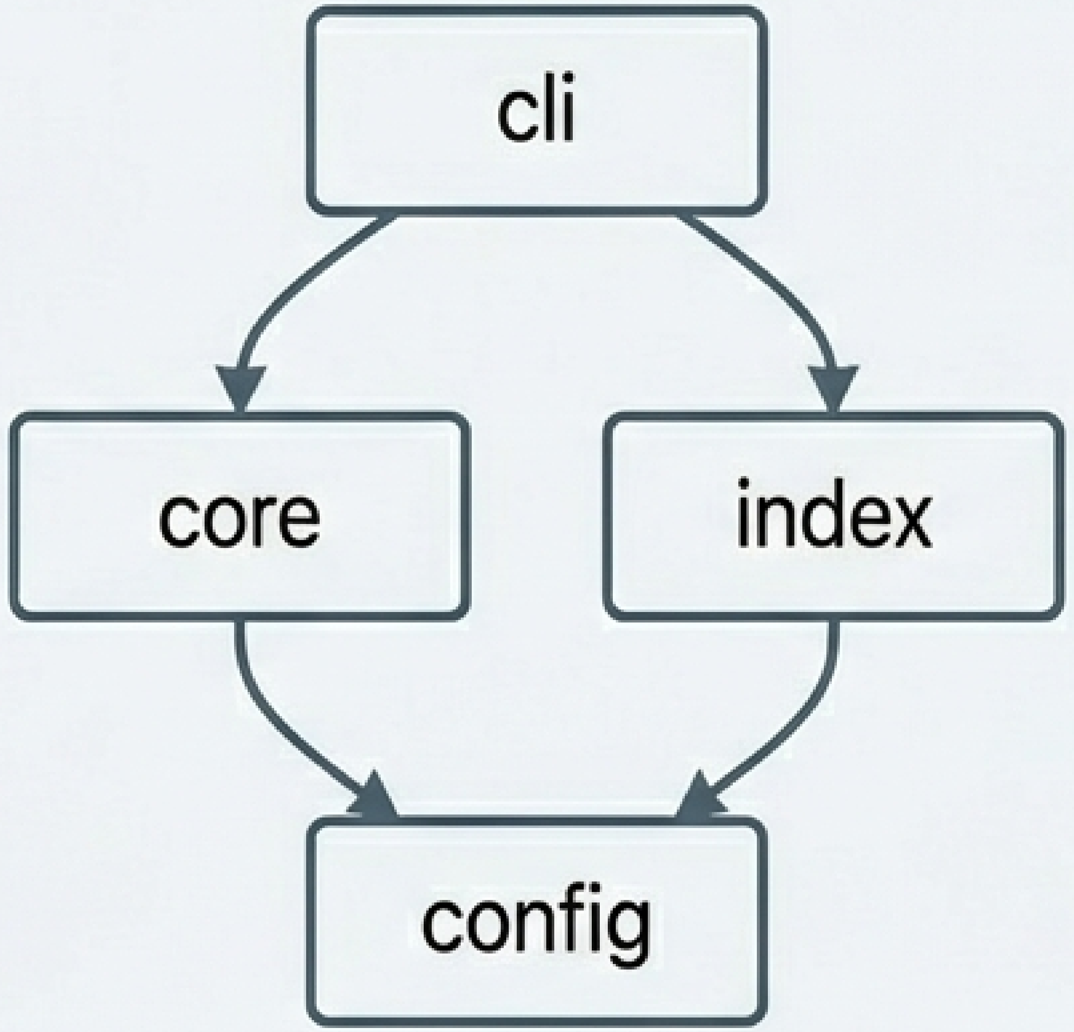
Tool 03: Graph (The Map)

Generating Mermaid, JSON, or DOT architecture visualizations.

GRAPH MODES

1. **'modules'**: Directory-level structure.
2. **'dependencies'**: Detailed code dependencies.
3. **'cross-repo'**: Microservice relationships.

VISUALIZATION (Mermaid Render)



Generated in ~0.2s via ‘modules’ mode.

Tool 04: Stats (The Pulse)

Instant workspace health and metadata verification.

TOTAL FILES

8,934

HEALTH STATUS

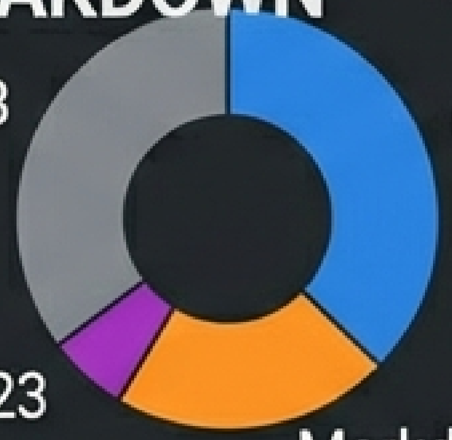
● healthy

INDEX SIZE

11.6 MB

Physical Disk Usage

FILE TYPE BREAKDOWN



Timestamps: lastScan: 2026-02-15T08:30:00Z

Client Integration

Configuring the Big 3 AI Agents.

CLAUDE CODE ~/.claude/config.json

```
"synthesis": {
  "command": "synthesis-mcp-server",
  "args": ["--workspace", "/path"]
}
```

CURSOR .cursor/mcp.json

```
"synthesis": {
  "command": "synthesis-mcp-server",
  "args": ["--workspace", "/path"]
}
```

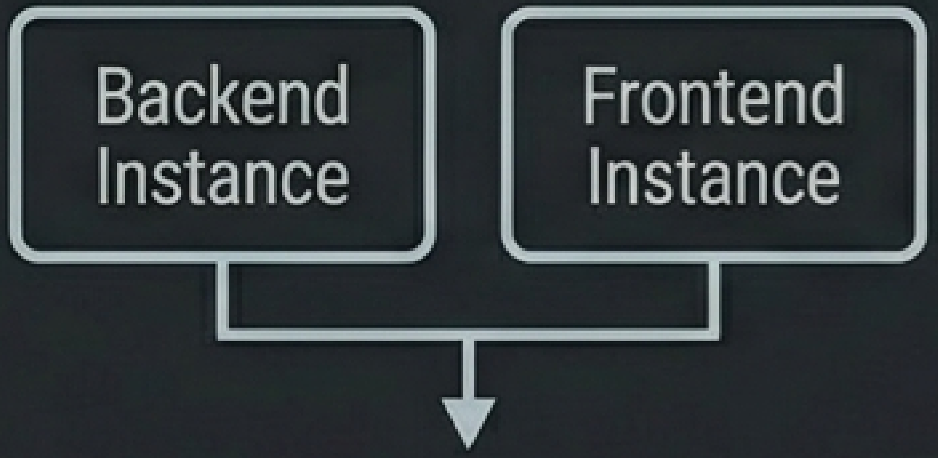
AIDER .aider.conf.yml

```
mcp-servers:
  - name: synthesis
    command: synthesis-mcp-server
```

Advanced Configuration

Tuning for Scale and Performance.

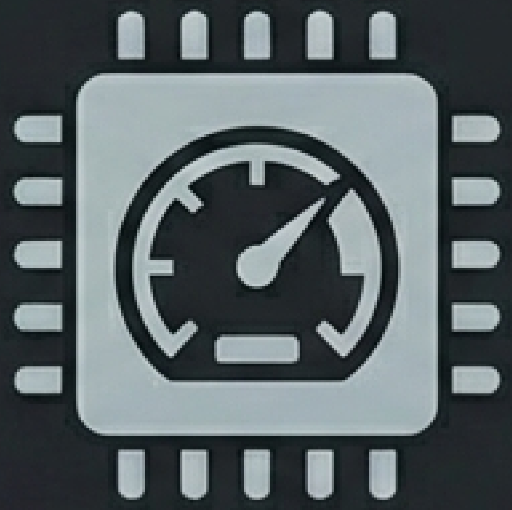
MULTIPLE WORKSPACES



Run separate instances for different contexts.

```
args: ['--workspace',
  '/src/backend']
```

PERFORMANCE TUNING



For workspaces > 50k files.

```
SYNTHESIS_JAVA_OPTS='-Xmx2g'
```

Run 'synthesis maintain' to compact index.

LOGGING

Debug JSON-RPC traffic.

```
args: ['--log-level', 'FINE']
```

~/.synthesis/logs/mcp-server.log

Real-World Workflows

From Theory to Practice.

SCENARIO A: SAFE REFACTORING

User Prompt "Rename TokenManager" → **Agent Action** Call "relate" → **Result** Identify 3 incoming references → **Outcome** 0% Breaking Changes

SCENARIO B: ONBOARDING

User Prompt "Explain Architecture" → **Agent Action** Call "graph" (modules) → **Agent Action** Call "search" (config) → **Outcome** Visual Walkthrough

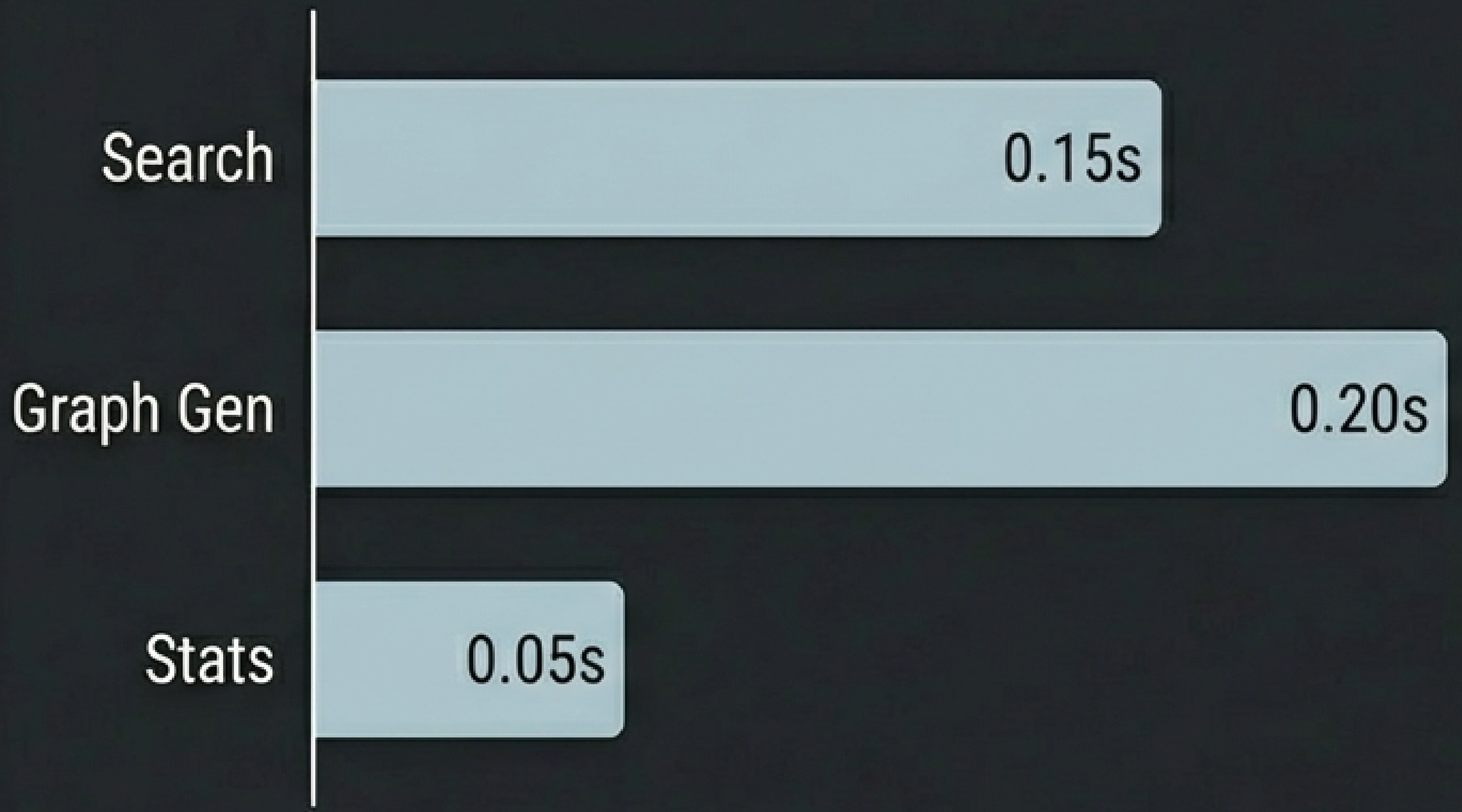
SCENARIO C: CROSS-REPO DISCOVERY

User Prompt "Find Payment Dependencies" → **Agent Action** Call "search" ("payment") → **Agent Action** Call "graph" (cross-repo) → **Outcome** Microservice Map

Performance Benchmarks

Test Environment: 16GB RAM / SSD / 8,934 Files.

Response Latency (Seconds)



DISCOVERY TIME

95%

Reduction vs grep/find

INDEX OVERHEAD

~5%

of Source Content Size

Search Time scales at O(log n) via Lucene Inverted Index.

Troubleshooting & Diagnostics

SYMPTOM: Java Error / Not Found

FIX: Install OpenJDK 17+. Verify with 'java -version'.

SYMPTOM: Not a Synthesis Workspace

FIX: Run 'synthesis init' and 'synthesis scan' in root.

SYMPTOM: Empty Search Results

FIX: Index stale? Run 'synthesis scan'. Check special char escaping in query.

SYMPTOM: JSON-RPC Errors (-32700)

FIX: Validate JSON syntax. Ensure 'jsonrpc: 2.0' is included.

Check detailed logs at ~/.synthesis/logs/mcp-server.log

Best Practices for Mastery

1. **KEEP IT FRESH**
   Always run 'synthesis scan' after major code changes (pulls, refactors).
2. **OPTIMIZE**
   Use 'synthesis maintain' periodically on large indexes to prevent fragmentation.
3. **SCOPE IT RIGHT**
   Use specific 'fileType' filters (e.g., fileType:CODE) to reduce token usage.
4. **VERIFY FIRST**
   Start complex sessions with 'stats' to ensure index is mounted and healthy.
5. **MEMORY MANAGEMENT**
   Increase heap size (-Xmx2g) if file count exceeds 50,000.

Empower Your AI with Context

```
synthesis-mcp-server --workspace /path/to/project
```

MCP Quick Start   Protocol Reference   GitHub Repository

"From blind guessing to structured understanding—Synthesis transforms your codebase into a queryable knowledge base."